



Informe sobre la inclusión de restricciones de un tipo de jornada de conductores en el algoritmo B&P

GITTY MELANY BACA VÁSCONEZ

18, MARZO 2026

Índice general

1. Motivación y limitaciones del modelo anterior	1
2. Metodología	2
2.1. Modelado de la jornada laboral mediante fases	2
2.2. Recursos RCSP y su correspondencia con restricciones laborales	3
2.2.1. Recurso continuo principal: tiempo de turno	3
2.2.2. Recurso continuo secundario: tiempo muerto	3
2.3. Estructura del grafo multi-capa	3
2.4. Función de costo y penalización de Huber	5
2.4.1. Variable de costo puro y el oráculo RCSP	6
3. Implementación	7
3.1. Nueva clase: <code>VSPRCSPSolver</code>	7
3.1.1. Inicialización de la red <code>VRPSolver</code>	7
3.1.2. Variables <code>tripVars</code> y <code>costVar</code>	8
3.1.3. Construcción de nodos: <code>createOrGetNode</code>	9
3.1.4. Construcción de vértices: <code>buildVertices()</code>	10
3.1.5. Construcción de arcos: <code>buildArcs()</code>	10
3.1.6. Creación del oráculo RCSP	12
3.2. Cambios en <code>VSPModel.cpp</code>	12
3.2.1. Delegación del subproblema a <code>VSPRCSPSolver</code>	12
3.2.2. Variables de holgura en el maestro	13
3.2.3. Restricciones de branching sobre <code>tripVars</code>	14
3.3. Nuevos parámetros de configuración de la aplicación	15
3.4. Configuración de <code>BaPCod</code> para <code>VRPSolver</code>	16
3.5. Resumen de cambios respecto al modelo anterior	21

3.6. Resultados Y Conclusiones	21
3.6.1. Instancias	22
3.6.2. Resultados	25
Bibliografia	28

RESUMEN

Este informe constituye la segunda entrega del proyecto de optimización de la flota de transporte público de la Empresa Pública Metropolitana de Transporte de Pasajeros de Quito (EPMTPQ). Partiendo del modelo de generación de columnas presentado en el informe anterior —que resolvía el Problema de Asignación de Vehículos con Múltiples Depósitos (MDVSP) sin restricciones laborales explícitas— se describe aquí la transición hacia un modelo que incorpora las regulaciones laborales de la jornada mediante un grafo multi-capas y un oráculo de caminos más cortos con restricciones de recursos (RCSP), implementado a través de la extensión VRPSolver de BaPCod. Se trabaja sobre instancias de Trolebús y Ecovía.

Palabras clave: MDVSP, grafo multi-capas, RCSP, BaPCod, VRPSolver, restricciones laborales, Branch-and-Price.

Capítulo 1

Motivación y limitaciones del modelo anterior

El modelo presentado en el informe anterior construía un grafo de conexión plano: un nodo fuente, un nodo sumidero y un nodo por cada viaje. Los arcos de conexión entre viajes se filtraban únicamente por compatibilidad geográfica (mismo terminal) y ventana de espera $[\text{min_espera}, \text{max_espera}]$. El costo de cada arco era el tiempo de espera entre viajes consecutivos, y el subproblema de pricing se resolvía como un programa lineal entero mediante Gurobi.

Esta formulación presentaba dos limitaciones fundamentales:

1. **Ausencia de restricciones laborales.** El grafo no distinguía en qué momento de la jornada se encontraba un conductor al realizar un viaje. Las regulaciones laborales aplicables —descanso obligatorio de 30 minutos tras el primer bloque de conducción, cambio de puesto de 20 minutos a mitad de la jornada, y un segundo descanso— no podían expresarse como restricciones del grafo.
2. **Ineficiencia del solver MIP para el pricing.** Al crecer el número de viajes y terminales, el solver MIP genérico se volvía el cuello de botella del algoritmo Branch-and-Price.

La solución adoptada, descrita en este informe, consiste en reemplazar el grafo plano por un *grafo multi-capas* y sustituir el solver MIP por el oráculo RCSP de VRPSolver.

Capítulo 2

Metodología

2.1. Modelado de la jornada laboral mediante fases

La jornada laboral de un conductor se divide en cuatro bloques o *fases*, separados por tres eventos obligatorios: dos descansos y un cambio de puesto. El Cuadro 2.1 resume las ventanas de tiempo acumulado asociadas a cada fase, expresadas en minutos desde el inicio del turno.

Cuadro 2.1: Definición de fases y ventanas de tiempo acumulado (minutos desde inicio de turno)

Fase	Mín.	Máx.	Descripción
F0	0	225	Primer bloque de conducción
F1	255	500	Segundo bloque, tras el primer descanso (30 min)
F2	510	735	Tercer bloque, tras el cambio de puesto (20 min)
F3	755	1020	Cuarto bloque, tras el segundo descanso (30 min)

Todos estos valores son configurables en `appParams.cfg` sin necesidad de recompilar.

La estrategia de modelado replica cada viaje en cuatro nodos, uno por fase, de modo que la posición de un nodo en el grafo codifica simultáneamente el viaje y el bloque laboral en curso.

2.2. Recursos RCSP y su correspondencia con restricciones laborales

El subproblema de pricing se formula como un Problema de Camino Más Corto con Restricciones de Recursos (RCSP). Se utilizan únicamente dos recursos continuos.

2.2.1. Recurso continuo principal: tiempo de turno

El recurso `RECURSO_TIEMPO_TURNO` acumula el tiempo transcurrido desde el inicio del turno, sumando la duración del viaje destino y el tiempo de espera en cada arco. Al declararse como recurso principal mediante `setAsMainResource()`, `VRPSolver` lo emplea para guiar el algoritmo de etiquetado. Las ventanas de consumo en cada vértice (Cuadro 2.1) actúan como restricciones de ventana de tiempo que descartan etiquetas inviables durante la expansión hacia adelante o hacia atrás.

2.2.2. Recurso continuo secundario: tiempo muerto

El recurso `RECURSO_TIEMPO_MUERTO` acumula el trabajo y las esperas acumuladas *antes* del viaje actual, sin incluir la duración de este último. Se declara como no desechable (*non-disposable*) mediante `setAsNonDisposableResource()`. Esto garantiza que si el acumulado no alcanza el límite inferior de la fase correspondiente, la ruta sea rechazada, en lugar de ajustar artificialmente el recurso hacia arriba como ocurriría con un recurso desechable.

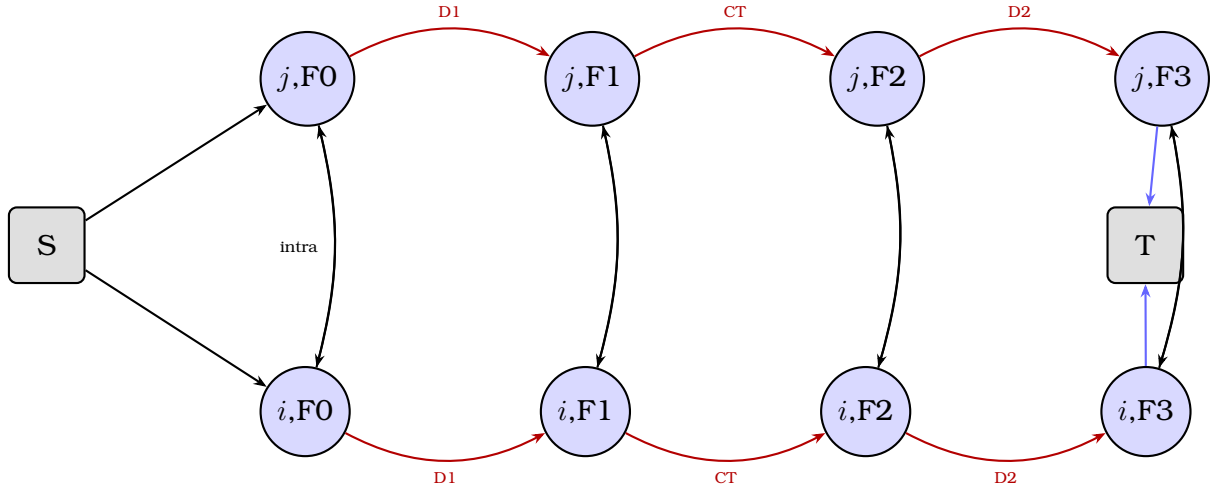
La distinción entre ambos recursos continuos responde a la necesidad de verificar tanto la cota superior ($\text{tiempo_turno} \leq \text{fase_X_max}$) como la cota inferior ($\text{tiempo_muerto} \geq \text{fase_X_min}$) de cada fase.

2.3. Estructura del grafo multi-capa

El grafo resultante, ilustrado en la Figura 2.1, tiene la siguiente estructura. Para cada viaje j que pasa por el terminal k , se crean cuatro nodos $(j, 0), (j, 1), (j, 2), (j, 3)$, uno por fase. Los arcos son de cinco tipos:

-
1. **Source** $\rightarrow$ **F0**: un arco por cada viaje que *parte* del terminal. Consume $\text{tiempo_turno} = \text{dur}_j$ y $\text{tiempo_muerto} = 0$.
 2. **Intra-fase**: conexiones entre viajes dentro de la misma fase cuando la espera está en $[\text{min_espera_intra_fase}, \text{max_espera_intra_fase}]$. El costo de arco es el tiempo muerto t_m .
 3. **Transición F0** $\rightarrow$ **F1**: requiere que el viaje origen *llegue* al terminal y el destino *salga* del terminal, con espera en $[\text{min_espera_transicion}, \text{max_espera_transicion}]$. El costo es la penalización de Huber respecto al tiempo ideal.
 4. **Transición F1** $\rightarrow$ **F2**: análogo al anterior, con penalización respecto al tiempo ideal de cambio de puesto.
 5. **Transición F2** $\rightarrow$ **F3**: análogo, con penalización respecto al tiempo ideal del segundo descanso.
 6. **F3** $\rightarrow$ **Sink**: arcos de retorno sin costo, únicamente desde nodos de fase 3 cuyo viaje *llega* al terminal k .

Dado que los arcos se crean únicamente cuando $t_{\text{inicio},j} \geq t_{\text{fin},i} + \text{min_espera}$, el tiempo fluye siempre hacia adelante y el grafo es un DAG. Esta propiedad garantiza que no pueden formarse ciclos dentro de ninguna fase, por lo que la secuencia de eventos laborales queda completamente determinada por la estructura de capas: la única forma de llegar a un nodo de fase $k + 1$ es atravesar un arco de transición desde la fase k , y la única forma de alcanzar el sumidero es completar las cuatro fases. Por este motivo, el modelo no requiere recursos adicionales para codificar los eventos laborales; las ventanas de tiempo de los dos recursos continuos son suficientes para garantizar la correctitud de toda ruta generada.



D1 = Descanso 1 **CT** = Cambio de Turno **D2** = Descanso 2
 → Arco intra-fase
 → Transición de fase (ventanas de recursos continuos)
 → F3 → T (único arco válido al sumidero)

Figura 2.1: Esquema simplificado del grafo multi-capas para dos viajes i y j .

2.4. Función de costo y penalización de Huber

A diferencia del modelo anterior, donde el costo de cada arco era simplemente el tiempo de espera, el nuevo modelo distingue dos tipos de costo:

Arcos intra-fase. El costo es el tiempo muerto t_m entre viajes consecutivos, acumulado en `costVar[0]` mediante `addVarAssociation`.

Arcos de transición entre fases. Se introduce una penalización basada en la función de Huber, que combina comportamiento lineal cerca del tiempo ideal y cuadrático lejos de él:

$$\text{pen}(t_m) = \begin{cases} |t_{\text{ideal}} - t_m| & \text{si } |t_{\text{ideal}} - t_m| \leq \delta \\ \delta + \frac{(|t_{\text{ideal}} - t_m| - \delta)^2}{\delta} & \text{en otro caso} \end{cases}$$

donde t_{ideal} es el tiempo de descanso o cambio de puesto objetivo y δ es la zona de confort. Los valores configurados son:

- F0→F1: $t_{\text{ideal}} = 30 \text{ min}$, $\delta = 8 \text{ min}$.

-
- $F1 \rightarrow F2$: $t_{\text{ideal}} = 20 \text{ min}$, $\delta = 5 \text{ min}$.
 - $F2 \rightarrow F3$: $t_{\text{ideal}} = 30 \text{ min}$, $\delta = 8 \text{ min}$.

Esta penalización incentiva que los descansos y cambios de puesto ocurran cerca del tiempo fijado, sin prohibir tiempos alternativos.

2.4.1. Variable de costo puro y el oráculo RCSP

Un aspecto técnico relevante de la extensión VRPSolver es la forma en que se comunica el costo acumulado al algoritmo de etiquetado. Según la documentación de BaPCod, cuando al menos un arco del grafo tiene un costo *puro* no nulo (es decir, un costo que no proviene del coeficiente de una variable del subproblema en la función objetivo), es necesario declarar una variable especial mediante `setPureCostBcVar()`. En esta implementación, `costVar[0]` cumple esa función: es una variable continua cuyo valor en cualquier solución del subproblema es igual a la suma de todos los tiempos muertos y penalizaciones de los arcos que forman el camino. La función objetivo del subproblema simplemente minimiza `costVar[0]`, y el costo fijo de ruta queda incorporado a nivel de BaPCod a través de `setFixedCost()`.

Capítulo 3

Implementación

3.1. Nueva clase: VSPRCSPSolver

La diferencia más importante respecto al modelo anterior es la aparición de la clase `VSPRCSPSolver`, declarada en `VSPRCSPSolver.h` e implementada en `VSPRCSPSolver.cpp`. Esta clase encapsula toda la lógica de construcción del grafo RCSP y expone un oráculo de tipo `BcRCSPFunctor` que BaP-Cod utilizará como solver del subproblema de pricing, sustituyendo completamente al solver MIP que se usaba en el modelo anterior.

Su constructor recibe la formulación del subproblema (`BcFormulation`), el identificador del terminal, la referencia a los datos globales y el array de restricciones de cobertura del maestro.

3.1.1. Inicialización de la red VRPSolver

El primer paso dentro del constructor es crear el grafo mediante la clase `BcNetwork` de `VRPSolver`:

```
1 BcNetwork network(this->spForm, data.n, 0, data.n);
2
3 BcNetworkResource tiempo_turno(network, RECURSO_TIEMPO_TURN0);
4 tiempo_turno.setAsMainResource();
5
6 BcNetworkResource tiempo_muerto(network, RECURSO_TIEMPO_MUERTO);
7 tiempo_muerto.setAsNonDisposableResource();
```

Listing 3.1: Declaración de recursos en la red

El constructor de `BcNetwork` recibe la formulación del subproblema y los números de conjuntos de elementaridad, empaquetamiento y cobertura. Según la firma `BcNetwork(bcForm, numElemSets, numPackSets, numCovSets)`, en esta llamada se pasan `data.n` conjuntos de elementaridad, 0 conjuntos de empaquetamiento y `data.n` conjuntos de cobertura.

Cada viaje j se asigna a su conjunto de elementaridad mediante `vertex.setPackingSet` garantizando que cada viaje sea servido a lo sumo una vez.

3.1.2. Variables `tripVars` y `costVar`

Se definen dos arrays de variables en el subproblema. El array `tripVars` contiene una variable binaria por viaje, y es el mecanismo mediante el cual la cobertura de viajes se comunica al problema maestro:

```
1 for (int j = 0; j < data.n; ++j) {
2     tripVars(j).type('B');
3     tripVars[j] >= 0.0;
4     tripVars[j] <= 1.0;
5     tripCoverage[j] += tripVars[j];
6 }
7 tripVars.priorityForMasterBranching(1.0);
8 tripVars.priorityForSubproblemBranching(0.0);
9 tripVars.priorityForRyanFosterBranching(0.0);
```

Listing 3.2: Definición de `tripVars` y enlace con restricciones maestras

La llamada `tripCoverage[j] += tripVars[j]` es la operación clave que une el subproblema con el maestro: indica a BaPCod que, cuando se selecciona un camino en el subproblema que visita el viaje j , la restricción de cobertura del viaje j debe ser satisfecha. Las prioridades de branching se configuran para que BaPCod realice branching sobre el maestro (`priorityForMasterBranching(1.0)`) sin intentar branching directo sobre las variables del subproblema ni Ryan-Foster, lo que es apropiado dado que cada subproblema tiene multiplicidad a lo sumo igual al número de buses del terminal.

La variable `costVar[0]` es una variable continua que acumula la suma de tiempos muertos y penalizaciones de los arcos del camino.

```
1 costVar(0).type('C');
2 costVar[0] >= 0.0;
```

```

3 BcObjective spObj(this->spForm);
4 spObj += costVar[0];

```

Listing 3.3: Definición de costVar y función objetivo del subproblema

3.1.3. Construcción de nodos: createOrGetNode

La función `createOrGetNode` implementa un patrón de creación diferida: recibe una fase y un identificador de viaje, y devuelve el ID del nodo correspondiente en la red `VRPSolver`, creándolo solo si no existía previamente. El mapa `nodeIdMap`, indexado por la clave `{fase, viaje_id}`, actúa como caché para evitar duplicados.

Una vez creado el vértice, se configuran las ventanas de consumo de los dos recursos continuos:

Ventanas de tiempo_turno. Este recurso acumula $t_m + dur_j$ al llegar al nodo j , es decir, el tiempo total transcurrido incluyendo la duración del viaje actual. Su cota inferior se fija en cero porque es un recurso disponible: el algoritmo de etiquetado puede ajustar el valor hacia arriba libremente si es necesario, por lo que imponer un mínimo no tiene efecto útil. La cota superior sí es relevante y varía por fase:

$$\text{tiempo_turno} \leq \text{fase_X_max}$$

Si el tiempo total acumulado al llegar al nodo j supera este límite, la etiqueta se descarta. Esto impide que un viaje se asigne a una fase cuando ya se ha superado su ventana temporal máxima.

Ventanas de tiempo_muerto. Este recurso acumula $t_m + dur_i$ al llegar al nodo j , es decir, el tiempo acumulado antes de comenzar el viaje actual, sin incluir su duración. Su cota superior se fija en `max_tiempo_ruta` porque no hay un límite superior operativo real. La cota inferior es la que impone la restricción de fase:

$$\text{tiempo_muerto} \geq \text{fase_X_min}$$

Al ser `tiempo_muerto` un recurso non-disposable, si el valor acumulado al llegar al nodo j no alcanza este mínimo, la etiqueta se rechaza directamente sin ningún ajuste. Esto garantiza que el conductor no puede entrar a una fase antes de haber acumulado el tiempo mínimo requerido.

La Tabla 3.1 resume los valores concretos configurados.

Conjunto de elementalidad. Finalmente, cada nodo de viaje recibe su conjunto mediante `vertex.setPackingSet(viaje_id)`. Todos los nodos $(j, 0), (j, 1), (j, 2), (j, 3)$ comparten el mismo `viaje_id`. El algoritmo de etiquetado de VRPSolver garantiza que ningún camino puede visitar dos nodos del mismo conjunto, lo que asegura que cada viaje aparece a lo sumo una vez en cualquier ruta sin necesidad de añadir una restricción explícita en el modelo maestro.

3.1.4. Construcción de vértices: `buildVertices()`

La función `buildVertices()` crea los nodos del grafo en el siguiente orden: primero el nodo fuente, luego el nodo sumidero, y finalmente los nodos de viaje en sus cuatro fases. La Tabla 3.1 resume las ventanas configuradas.

Cuadro 3.1: Ventanas de recursos continuos por fase

Fase	tiempo_turno LB	tiempo_turno UB	tiempo_muerto LB	tiempo_muerto UB
F0	0	225	0	1020
F1	0	500	255	1020
F2	0	735	510	1020
F3	0	1020	755	1020
Source	0	0	0	0
Sink	0	1020	0	1020

3.1.5. Construcción de arcos: `buildArcs()`

La función `buildArcs()` construye los cinco tipos de arcos descritos en la sección de metodología.

Arcos Source $\rightarrow$ F0. Solo se crean para viajes que parten del terminal k . El consumo del recurso de tiempo es la duración del viaje, y el arco queda asociado a la variable `tripVars[j]` mediante `arc.arcVar()`:

```

1      int dur_j = data.viajes[j].fin - data.viajes[j].inicio;
2      BcArc arc = network.createArc(sourceId, it->second, 0.0);
3      tiempo_turno.setArcConsumption(arc, dur_j);
4      tiempo_muerto.setArcConsumption(arc, 0.0);
5      arc.arcVar(tripVars[j]);

```

Listing 3.4: Creación de arcos Source a Fase 0

Arcos intra-fase. Se iteran todas las fases y todos los pares de viajes compatibles geográficamente:

```
1  int dur_i = data.viajes[i].fin - data.viajes[i].inicio;
2  int dur_j = data.viajes[j].fin - data.viajes[j].inicio;
3  BcArc arc = network.createArc(it_i->second, it_j->second, 0.0);
4  tiempo_turno.setArcConsumption(arc, tm + dur_j);
5  tiempo_muerto.setArcConsumption(arc, tm + dur_i);
6  arc.arcVar(tripVars[j]);
7  arc.addVarAssociation(costVar[0], tm);
```

Listing 3.5: Arcos intra-fase con costo de tiempo muerto

Arcos de transición entre fases. Se ilustra el caso $F0 \rightarrow F1$; los demás son análogos:

```
1  double pen = penHuber(data.pen_ideal_f0_f1, tm, 8.0);
2  int dur_i = data.viajes[i].fin - data.viajes[i].inicio;
3  int dur_j = data.viajes[j].fin - data.viajes[j].inicio;
4  BcArc arc = network.createArc(it_i->second, it_j->second, 0.0);
5  tiempo_turno.setArcConsumption(arc, tm + dur_j);
6  tiempo_muerto.setArcConsumption(arc, tm + dur_i);
7  arc.arcVar(tripVars[j]);
8  arc.addVarAssociation(costVar[0], pen);
```

Listing 3.6: Arco de transición $F0 \rightarrow F1$ con penalización de Huber

donde la función de Huber es:

```
1  auto penHuber = [](double ideal, double tm, double zona) -> double
2  {
3      double d = std::fabs(ideal - tm);
4      if (d <= zona)
5          return d;
6      else
7          return zona + (d - zona) * (d - zona) / zona;
8  };
```

Listing 3.7: Implementación de la penalización de Huber

Arcos al sumidero. Se crean desde los nodos de la fase 3 cuyo viaje llega al terminal k , con consumo de tiempo cero y sin costo asociado.

```
1      BcArc arc = network.createArc(e.second, sinkId, 0.0);
2      tiempo_turno.setArcConsumption(arc, 0.0);
3      tiempo_muerto.setArcConsumption(arc, 0.0);
```

Listing 3.8: Creación de arcos Fase 3 a Sink

3.1.6. Creación del oráculo RCSP

```
1  #ifndef BCP_RCSP_IS_FOUND
2      oracle = new BcRCSPFunctor(this->spForm);
3      oracle->setPureCostBcVar(costVar[0]);
4  #endif
```

Listing 3.9: Instanciación del oráculo BcRCSPFunctor

BcRCSPFunctor implementa el algoritmo de etiquetado con grafos de cubos (*bucket-graph based labeling algorithm*). Al llamar a `setPureCostBcVar(costVar[0])`, se comunica al algoritmo que el valor de esta variable en cualquier solución es igual a la suma de los costos puros de los arcos del camino.

3.2. Cambios en VSPModel.cpp

3.2.1. Delegación del subproblema a VSPRCSPSolver

En el modelo anterior, el subproblema se definía directamente en `buildVSPModel()` mediante variables y restricciones de flujo de red. En la nueva versión, toda esa lógica se delega a la clase `VSPRCSPSolver`:

```
1 BcFormulation subProb = colGenSp(k);
2 int max_buses = data.terminals[k].num_buses > 0
3     ? data.terminals[k].num_buses : 1000;
4 subProb <= max_buses;
5 subProb.setFixedCost(data.costo_fijo_ruta);
6
7 VSPRCSPSolver* solver = new VSPRCSPSolver(
8     subProb, data.terminals[k].id, data, tripCoverage
```

```

9 );
10 #ifdef BCP_RCSP_IS_FOUND
11     subProb.attach(solver->getOracle());
12 #endif

```

Listing 3.10: Instanciación de VSPRCSPSolver por terminal

La llamada `subProb.setFixedCost()` incorpora el costo fijo de ruta a nivel de BaPCod, de modo que cada columna generada lleva automáticamente ese costo adicional en la función objetivo del maestro. Esto simplifica el subproblema, que solo necesita minimizar tiempos muertos y penalizaciones.

3.2.2. Variables de holgura en el maestro

En la formulación estándar del MDVSP con generación de columnas, las restricciones de cobertura se definen como igualdades:

$$\sum_r a_{ir} y_r = 1 \quad \forall i \in \mathcal{V}$$

donde $a_{ir} = 1$ si la ruta r cubre el viaje i , e y_r es la variable de decisión asociada a la ruta. Esta formulación exige que todo viaje sea cubierto exactamente una vez. Sin embargo, con restricciones laborales explícitas, existen viajes que no pueden ser asignados a ninguna ruta factible bajo las ventanas de fase configuradas. Mantener la igualdad estricta provoca que BaPCod declare infactibilidad antes de siquiera completar la generación de columnas. Para resolverlo, se relaja la restricción a una desigualdad y se añade una variable de holgura binaria s_i por viaje:

$$\sum_r a_{ir} y_r + s_i \geq 1 \quad \forall i \in \mathcal{V}, \quad s_i \in \{0, 1\}$$

La implementación en `VSPModel.cpp` es la siguiente:

```

1 BcVarArray slackVar(master, "SlackNoCubierto");
2 for(int i = 0; i < data.n; ++i) {
3     slackVar(i).type('B');
4     slackVar[i] >= 0.0;
5     slackVar[i] <= 1.0;
6     tripCoverage(i) == 1.0;

```

```

7     tripCoverage(i) += slackVar[i];
8 }
9 BcObjective masterObj(master);
10 for(int i = 0; i < data.n; ++i)
11     masterObj += data.costo_columna_artificial * slackVar[i];

```

Listing 3.11: Variables de holgura en el maestro

Esta formulación tiene tres efectos concretos. Primero, el maestro restringido es siempre factible desde el inicio: al arrancar sin ninguna columna real generada, basta con fijar $s_i = 1$ para todo viaje y la restricción se satisface. Segundo, el costo elevado `costo_columna_artificial` en la función objetivo empuja al solver a reemplazar cada holgura activa por una ruta real siempre que exista una, de modo que los viajes que sí tienen rutas factibles quedan cubiertos por columnas generadas. Tercero, los viajes que genuinamente no admiten ninguna ruta factible bajo las restricciones laborales terminan con $s_i = 1$ en la solución final, lo que permite identificarlos explícitamente en lugar de obtener un estado de infactibilidad que no aporta información.

La variable se declara binaria y no continua porque la restricción es ≥ 1 y la cobertura mínima requerida es entera: o el viaje está cubierto ($s_i = 0$) o no lo está ($s_i = 1$). Una holgura continua podría tomar valores fraccionarios en la relajación LP, complicando la interpretación de la solución.

3.2.3. Restricciones de branching sobre tripVars

La nueva versión añade un mecanismo de branching sobre restricciones explícito mediante `BcBranchingConstrArray`. En cada iteración, para cada viaje j , se acumula en la restricción de branching j la suma de las variables `tripVars[j]` de todos los subproblemas. Cuando BaPCod detecta que el valor fraccional de una de estas sumas no es entero, genera dos nodos hijos con la restricción $\leq \lfloor v \rfloor$ y $\geq \lceil v \rceil$ respectivamente:

```

1 BcBranchingConstrArray tripBranching(
2     master, "TripBranching",
3     SelectionStrategy::MostFractional, 2.0
4 );
5 for (int k = 0; k < data.d; ++k) {
6     BcFormulation subProb = colGenSp[k];

```

```

7   BcVarArray tripVars(subProb, "tripVar");
8   for (int j = 0; j < data.n; ++j)
9       tripBranching(j) += tripVars[j];
10 }

```

Listing 3.12: Branching constraints por viaje

La estrategia `MostFractional` prioriza el branching sobre la variable con valor más cercano a 0.5, lo que suele producir los árboles de búsqueda más equilibrados. La prioridad 2.0 es superior a la prioridad por defecto del branching sobre variables (1.0), asegurando que este mecanismo se aplique primero.

3.3. Nuevos parámetros de configuración de la aplicación

El archivo `appParams.cfg` fue ampliado con todos los parámetros necesarios para el grafo multi-capa. Estos parámetros son cargados por `VSPData::leerConfigApp()` y utilizados directamente por `VSPRCSPSolver`. La Tabla 3.2 resume los grupos de parámetros añadidos.

Cuadro 3.2: Nuevos grupos de parámetros en `appParams.cfg`

Grupo	Parámetros	
Flexibilidad de fases	alpha_flexibilidad, alpha_primer_turno	
Límites de fases	fase_0_min/max, fase_2_min/max, max_tiempo_ruta	fase_1_min/max, fase_3_min/max,
Esperas intra-fase	min_espera_intra_fase, max_espera_intra_fase	
Esperas de transición	min_espera_transicion, max_espera_transicion	
Penalizaciones ideales	pen_ideal_f0_f1, pen_ideal_f2_f3	pen_ideal_f1_f2,

Centralizar estos parámetros en el archivo de configuración, en lugar de codificarlos como constantes en el código fuente, permite ajustar la estructura del grafo ante cambios regulatorios o variaciones de instancia sin necesidad de recompilar.

3.4. Configuración de BaPCod para VRPSolver

La Tabla 3.3 recoge los parámetros configurados en `bcParameters.cfg` para el modelo actual. Se organizan por categoría para facilitar su interpretación.

Cuadro 3.3: Parámetros de BaPCod configurados para la extensión VRPSolver, instancia C1

Parámetro	Valor	Descripción
<i>Parámetros principales (3.1)</i>		
<code>GlobalTimeLimit</code>	18000	Límite de tiempo en segundos para resolver el modelo (5 horas)
<code>MaxNbOfBBtreeNodeTreated</code>	5000	Número máximo de nodos explorados en el árbol B&B
<code>treeSearchStrategy</code>	0	Estrategia <i>best-first</i> para el árbol primario
<code>OpenNodesLimit</code>	500	Número máximo de nodos abiertos en la estrategia <i>best-first</i>
<i>Generación de columnas (3.2 y 6.4)</i>		
<code>colGenSubProbSolMode</code>	3	Resolver el pricing con el functor definido por el usuario (VRPSolver)
<code>mastInitMode</code>	3	Variables artificiales locales para inicializar el maestro restringido
<code>MaxNbOfStagesInColGenProcedure</code>	3	Número de fases de generación de columnas

Continúa en la página siguiente...

Parámetro	Valor	Descripción
GenerateProperColumns	true	Genera solo columnas que respetan cotas de variables del subproblema
RCSPmaxNumOfColsPerExactIteration	150	Máx. columnas por iteración en fase exacta
RCSPmaxNumOfColsPerIteration	30	Máx. columnas por iteración en fases heurísticas
RCSPstopCutGenTimeThresholdInPricing	10	Umbral de tiempo suave para el algoritmo de etiquetado, en segundos
RCSPhardTimeThresholdInPricing	20	Umbral de tiempo duro para el algoritmo de etiquetado, en segundos
RCSPuseBidirectionalSearch	0	Búsqueda bidireccional no usada
RCSPnumberOfBucketsPerVertex	25	Parámetro para el cálculo del tamaño de paso de los cubos
RCSPdynamicBucketSteps	1	Ajuste dinámico del tamaño de paso activado
RCSPapplyReducedCostFixing	1	Procedimiento estándar de eliminación de arcos de cubo
ReducedCostFixingThreshold	0.9	Frecuencia con la que se llama al procedimiento de eliminación de arcos
<i>Enumeración de rutas</i>		
RCSPmaxNumOfLabelsInEnumeration	0	Máx. etiquetas en la enumeración de rutas elementales
RCSPmaxNumOfEnumeratedSolutions	0	Máx. rutas enumeradas

Continúa en la página siguiente...

Parámetro	Valor	Descripción
RCSPmaxNumOfEnumSolutionsForMIP	0	Máx. rutas enumeradas para activar el MIP enumerado
<i>Cortes Rank-1 (6.4)</i>		
RCSPrankOneCutsMaxNumPerRound	100	Máx. cortes lm-R1C por ronda de separación
RCSPrankOneCutsMaxNumRows	5	Máx. filas por corte lm-R1C
RCSPrankOneCutsMemoryType	2	Tipo de memoria: memoria de nodo
<i>Condición de tailing-off en cortes (3.3)</i>		
CutTailingOffThreshold	0.01	Si la mejora relativa del gap es menor a este valor, se incrementa el contador de tailing-off
CutTailingOffCounterThreshold	3	Valor del contador para activar la condición de tailing-off
<i>Strong branching simplificado (3.6)</i>		
SimplifiedStrongBranchingParameterisation	true	Activa la parametrización simplificada de strong branching
StrongBranchingPhaseOneCandidatesNumber	10	Máx. candidatos evaluados en la fase 1
StrongBranchingPhaseOneTreeSizeEstimRatio	0.3	Ratio del tamaño del árbol para la fase 1
StrongBranchingPhaseTwoCandidatesNumber	3	Máx. candidatos evaluados en la fase 2
StrongBranchingPhaseTwoTreeSizeEstimRatio	0.1	Ratio del tamaño del árbol para la fase 2
<i>Heurísticas primales (3.5)</i>		

Continúa en la página siguiente...

Parámetro	Valor	Descripción
MaxTimeForRestrictedMasterIpHeur	-1	Tiempo máximo en segundos para el MIP restringido; valor negativo desactiva la heurística
CallFrequencyOfRestrictedMasterIpHeur	1	Frecuencia de llamada de la heurística restringida
RoundingColSelectionCriteria	4 6 9	Criterios de selección de columnas para redondeo
MIPemphasisInRestrictedMasterIpHeur	1	Énfasis del solver MIP en la heurística restringida
DivingHeurUseDepthLimit	30	Profundidad máxima en el árbol para la heurística de diving
CallFrequencyOfDivingHeur	1	Frecuencia de llamada de la heurística de diving
FixIntValBeforeRoundingHeur	true	Fija columnas con valor entero antes de redondear
MaxNbOfCgIteDuringRh	100	Límite de iteraciones de generación de columnas en cada nodo del diving
<i>Estabilización por suavizado de precios duales (3.4)</i>		
colGenDualPriceSmoothingAlphaFactor	1.0	Parámetro α del suavizado de Wentges; valor 1.0 activa el ajuste automático
colGenDualPriceSmoothingBetaFactor	0.0	Parámetro β del suavizado direccional; valor 0.0 desactiva la técnica

Los parámetros más relevantes para comprender la configuración actual son los siguientes.

Modo de pricing (3.2). `colGenSubProbSolMode = 3` indica a BaPCod que el subproblema de pricing se resuelve mediante el functor definido por el usuario. El modelo anterior usaba el valor 2 (solver MIP genérico).

Estrategia de búsqueda (3.1). `treeSearchStrategy = 0` corresponde a la estrategia *best-first* para el árbol primario, en la que el nodo abierto con menor cota inferior se considera primero. El parámetro `OpenNodesLimit = 500` limita el número máximo de nodos en el árbol primario; cuando se alcanza, los nuevos nodos se trasladan al árbol secundario, que siempre se explora en modo *depth-first*.

Búsqueda bidireccional desactivada (6.4). `RCSPuseBidirectionalSearch = 0` corresponde al valor $\varphi^{\text{bidir}} = 0$, que según la documentación significa que la búsqueda bidireccional no es usada. Los valores posibles son: 0 (no usada), 1 (siempre usada), 2 (usada solo en la fase exacta de generación de columnas).

Enumeración de rutas desactivada (6.4). Los parámetros ω^{labels} , ω^{routes} y ω^{MIP} en cero desactivan completamente el procedimiento de enumeración de rutas elementales y el MIP enumerado.

Heurística de diving activa (3.5). `DivingHeurUseDepthLimit = 30` activa la heurística de diving hasta una profundidad de 30 nodos en el árbol. La heurística se llama con frecuencia 1 (`CallFrequencyOfDivingHeur = 1`), es decir, en cada nodo del árbol de Branch-and-Bound. La selección de columnas para redondeo sigue los criterios 4, 6 y 9: primero menor distancia al entero más cercano no nulo, luego valor más cercano a su redondeo superior, y finalmente menor costo de columna.

Heurística MIP restringida desactivada (3.5).

`MaxTimeForRestrictedMasterIpHeur = -1` desactiva la heurística basada en resolver el maestro restringido como MIP.

Estabilización (3.4). `colGenDualPriceSmoothingAlphaFactor = 1.0` activa el suavizado de Wentges con ajuste automático del parámetro α . El suavizado direccional queda desactivado con $\beta = 0$.

3.5. Resumen de cambios respecto al modelo anterior

La Tabla 3.4 sintetiza las diferencias estructurales entre la implementación anterior y la actual.

Cuadro 3.4: Comparación entre modelo anterior y modelo con grafo multi-capas

Aspecto	Modelo anterior	Modelo actual
Grafo de subproblema	Plano (source, viajes, sink)	Multi-capas: 4 fases por viaje
Solver de pricing	MIP genérico (Gurobi/CPLEX)	Functor RCSP VRPSolver
Recursos del grafo	Ninguno	2 recursos continuos (main + non-disposable)
Restricciones laborales	—	Explícitas (recursos non-disposable)
Costo de arcos	Solo tiempo muerto	Tiempo muerto + penalizaciones de transición
Costo fijo de ruta	En función objetivo del subproblema	Vía <code>setFixedCost()</code> en formulación
Branching	Sobre variables de subproblema	<code>BcBranchingConstrArray</code> sobre <code>tripVars</code>
Archivo nuevo	—	<code>VSPRCSPSolver.h/.cpp</code>

3.6. Resultados Y Conclusiones

Los experimentos se realizaron en una computadora con sistema operativo Windows 11, 16 GB de RAM, procesador Intel Core i7-1255U de 12.^a generación (10 núcleos físicos). El código puede encontrarse en: https://github.com/LuisMiguelT/GenCol-VSP/tree/main/src/Melany/Bapcod_modelo_con_restricciones

3.6.1. Instancias

C1 y C2 del Trolebús de Quito

La instancia corresponde a la operación diaria (lunes a viernes) de las rutas C1 y C2 del Trolebús de Quito. La ruta C1 conecta el terminal El Recreo (sur) con el terminal El Labrador (norte), mientras que la ruta C2 extiende el servicio desde el terminal Morán Valverde hasta El Labrador. La Tabla 3.5 resume las características principales de la instancia.

Cuadro 3.5: Características de la instancia C1+C2 del Trolebús de Quito

Parámetro	Valor
Número de depósitos	3
Terminal El Recreo	27 buses
Terminal El Labrador	18 buses
Terminal Morán Valverde	14 buses
Flota total	59 buses
Total de viajes	842
Recreo → Labrador (C1)	372
Labrador → Recreo (C1)	372
Morán Valverde → Labrador (C2)	49
Labrador → Morán Valverde (C2)	49
Ventana de operación	05:00 – 22:46
Duración del viaje C1	45–48 min
Duración del viaje C2	63–65 min
<i>Restricciones laborales</i>	
Tiempo mínimo de conexión entre viajes	2 min

Una característica particular de esta instancia es la presencia de rutas *mixtas*: los 9 buses del tipo M, con base en Morán Valverde, operan viajes C2 en horas pico y viajes C1 (Recreo ↔ Labrador) durante el horario valle del mediodía, lo que exige que el modelo permita transiciones entre ambas rutas dentro de una misma jornada.

Como referencia, la Figura 3.1 muestra la asignación operativa manual actualmente en uso, la cual cubre la totalidad de los 842 viajes distribuidos entre los 59 vehículos. Sin embargo, esta solución no respeta las restriccio-

nes laborales: no garantiza los tiempos mínimos de descanso ni los límites de jornada exigidos por la normativa vigente. El objetivo del modelo de optimización es obtener una solución que cubra el mayor número posible de viajes *cumpliendo* dichas restricciones.



Figura 3.1: Solución de referencia manual para la instancia C1+C2. Cada fila representa un vehículo. Verde: tiempo en servicio; gris: tiempo muerto ≤ 30 min; rojo: tiempo muerto > 30 min.

3.6.2. Resultados

C1 y C2 del Trolébus de Quito

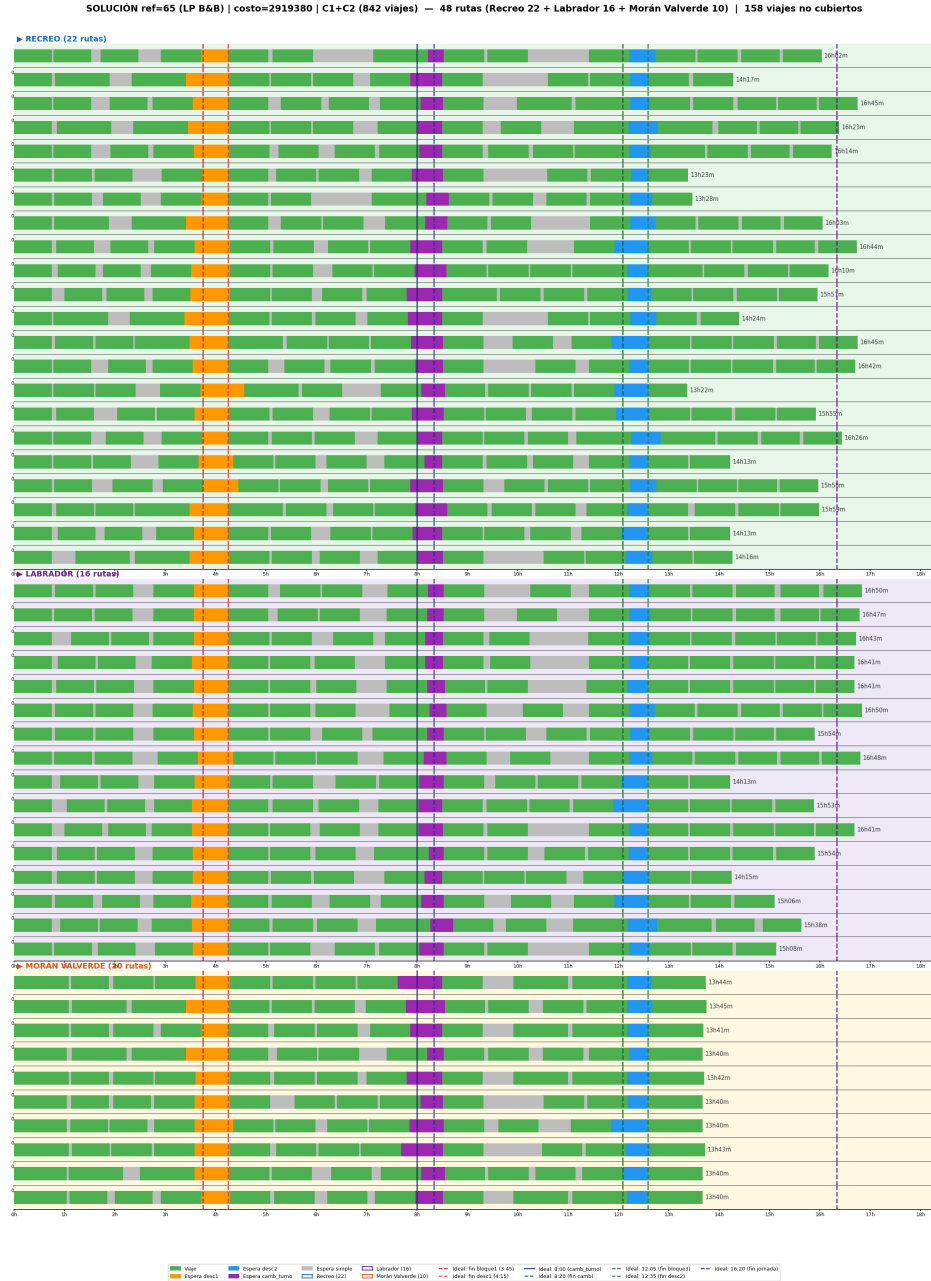


Figura 3.2: Solución obtenida con el modelo para la instancia C1+C2. Cada fila representa un vehículo.

Los experimentos se realizaron sobre la instancia descrita en la sección anterior. La Tabla 3.6 presenta los principales indicadores de rendimiento obtenidos.

Cuadro 3.6: Resultados del Branch-and-Price para la instancia C1+C2

Indicador	Valor
Mejor cota primal (PB)	2 919 380.32
Cota dual final (DB)	898 282
Gap de optimalidad	69.2 %
Tiempo de cómputo	18 000 s (5 h)
Viajes cubiertos	684 / 842
Viajes no cubiertos	158
Rutas generadas	48
Depósito El Recreo	22
Depósito El Labrador	16
Depósito Morán Valverde	10
Rutas mixtas C1+C2	28
Depósito El Recreo	15
Depósito El Labrador	3
Depósito Morán Valverde	10
Rutas puras C1	20
Rutas puras C2	0

El algoritmo Branch-and-Price obtuvo una solución que cubre 684 de los 842 viajes (81.2 %) respetando íntegramente las restricciones laborales, con un total de 48 rutas asignadas. Los 158 viajes no cubiertos representan demanda que supera la capacidad simultánea disponible de la flota bajo las restricciones impuestas.

Un resultado destacable es la generación automática de 28 rutas mixtas C1+C2, es decir, rutas en las que un mismo vehículo realiza viajes de ambas líneas dentro de una misma jornada. Las 10 rutas del depósito Morán Valverde son todas mixtas, pero el modelo extiende este comportamiento también a 15 rutas del depósito El Recreo y 3 rutas del depósito El Labrador. Ninguna ruta es puramente C2, lo que indica que el modelo aprovecha los viajes C2 intercalándolos con viajes C1 para maximizar la utilización de cada vehículo.

Un análisis de los viajes no cubiertos revela que la distribución no es uniforme a lo largo del día: la franja horaria de 17:00–19:00 concentra el mayor

número de viajes no cubiertos (31), seguida por las franjas de mañana temprana (07:00–09:00 y 09:00–11:00) con 26 y 25 viajes respectivamente. Estas franjas corresponden a las horas punta del sistema, donde la demanda simultánea supera la capacidad instalada en los terminales.

Respecto a la calidad de la solución, el gap de optimalidad del 69.2 % indica que el solver no pudo probar la optimalidad dentro del límite de tiempo de 5 horas. Esto se debe principalmente a la complejidad combinatoria del problema: la instancia cuenta con 842 viajes, 3 depósitos heterogéneos y restricciones laborales que acotan fuertemente el espacio de soluciones factibles. No obstante, la solución obtenida cubre el 81.2 % de los viajes sin violar ninguna restricción operativa, lo que representa una mejora significativa respecto a la solución manual de referencia que, si bien cubre todos los viajes, no satisface las restricciones laborales vigentes.

Referencias bibliográficas
